

Day 10 — Checker Types, Stimulus, and Priority

Checker/stimulus/priority mapping for the FIFO, traffic-light assertions, the FIFO scoreboard, and the directed-vs-random rationale.

Note on scope: using the refined 8-feature list, not the original 5

The assignment says “for each of the 5 FIFO features you identified” — that was true as of Day 3, but Day 9’s refinement pass split one over-broad feature in two and found a missing symmetric feature, bringing the real, current list to 8. Per Learning Topic 9.1 (“the verification plan is a living document — updated as ... design changes are made”), working from the refined list is the right call, not the outdated one.

Items 1 & 4 — Checker Type, Stimulus, and Priority for all 8 FIFO Features

Combined into one table since the same 8 rows apply to both — this is also exactly the Output table the assignment asks for.

Feature	Req-ID	Checker Type	Stimulus	Priority	Justification
reset_initializes_fifo_to_empty	RST_001	Checker w/ expected value	Directed	P0	Post-reset values are fully known in advance and don't depend on prior state — a simple expected-value check is sufficient. P0 because every other test depends on starting from a known-good state.
write_increments_count_and_stores_data_correctly	FUNC_001	Scoreboard	Constrained-random	P0	Data correctness over many operations is exactly what a scoreboard is for; random values exercise far more of the data space than a few hand-picked ones. P0 — without correct writes the FIFO does nothing useful.
read_returns_data_in_fifo_order	FUNC_002	Scoreboard	Constrained-random	P0	Order preservation across many operations is a data-integrity property; random interleavings stress this harder than one fixed sequence. P0 — a FIFO that doesn't preserve order isn't a FIFO.
full_flag_asserts_when_count_equals_depth	CORNER_001	Checker w/ expected value	Directed	P0	The exact cycle <code>full</code> should assert is fully predictable; directed stimulus <i>guarantees</i> the test actually reaches <code>count==DEPTH</code> rather than hoping random stimulus gets there. Explicitly named as a P0 example in 10.3.
write_blocked_when_full	NEG_001	Assertion	Directed	P0	" <code>count</code> must never exceed <code>DEPTH</code> " is a protocol invariant that should hold every cycle, not just one checked moment — that's what assertions are for. Directed stimulus is needed to reliably construct the full condition first. P0 — a leaking full flag makes real data loss/corruption possible.
empty_flag_asserts_when_count_equals_zero	CORNER_002	Checker w/ expected value	Directed	P0	Symmetric to <code>CORNER_001</code> .
read_blocked_when_empty	NEG_002	Assertion	Directed	P0	Symmetric to <code>NEG_001</code> — " <code>count</code> must never underflow below 0."
count_unchanged_on_simultaneous_read_write	CORNER_003	Assertion	Directed	P1	Whether <code>count</code> stays unchanged during simultaneous access is a protocol invariant, checkable continuously; directed stimulus is still needed to reliably construct the tricky full/empty sub-cases. P1 (not P0) because it's explicitly named as a P1 example in 10.3 — an important interaction, but a design that avoided ever asserting both signals together wouldn't be fundamentally broken the way a missing reset would be.

Item 2 — Three Plain-English Assertions: Traffic Light Controller

1. `main_green` and `side_green` must never be HIGH at the same time.
2. Exactly one of `{main_red, main_yellow, main_green}` must be HIGH at any given time — never zero, never two — and the same holds independently for `{side_red, side_yellow, side_green}`.
3. Whenever `main_yellow` is HIGH, `side_red` must also be HIGH, and whenever `side_yellow` is HIGH, `main_red` must also be HIGH. (This is the explicit “other road stays red during a yellow phase” design decision documented back in Day 5/7 — turning it into an assertion means it’s now checked continuously, not just assumed from reading the RTL comment.)

Item 3 — FIFO Scoreboard, Described in Plain English

What it stores: its own simple software queue (a plain array or list — not hardware) that mirrors every value the testbench writes into the DUT, in the exact order those writes happen. This is the “reference model” — simple enough to trust precisely because it’s separate from, and much simpler than, the DUT itself.

When it compares: every time the testbench asserts `rd_en` on a cycle where the DUT’s `empty` flag is LOW (i.e., a real read is actually expected to happen), the scoreboard pops the oldest entry off its own queue and compares it against whatever the DUT’s `dout` shows.

What a mismatch means: a genuine data-integrity failure — the DUT returned the wrong value, returned data out of order, corrupted something internally, or (less commonly) the scoreboard’s own model of “what should happen” has drifted out of sync with the DUT (e.g., it mistakenly treated a blocked read as a real one).

Either way, a mismatch is a hard failure requiring investigation — not something to average away or dismiss as noise. **Even one mismatch means something about data integrity is broken.**

Item 5 — Why Use Both Directed and Random Stimulus on the Same Design?

Directed and random stimulus solve two different problems, and neither one alone gets you all the way to confident sign-off. Directed stimulus is what you use when you need to *guarantee* a specific scenario actually happens — reset sequences, hitting an exact boundary like `count==DEPTH`, or engineering the precise setup needed for a negative test like “write while full” — because random stimulus might take a long time (or never, within a reasonable simulation budget) to stumble into that exact condition on its own. Random stimulus is what you use to find the bugs you didn’t think to write a directed test for in the first place — unexpected interactions between signals, data-integrity issues that only show up after many operations in an order nobody specifically planned, or timing combinations that weren’t obvious during spec-writing. Using only directed tests caps your confidence at your own imagination — you only verify the scenarios you thought of. Using only random tests means you might never reliably hit the specific corner cases that matter most, and gives you no efficient way to check simple, fully-predictable outcomes without a full scoreboard. In practice, directed tests carry the P0 corner cases and reset/negative scenarios where predictability matters most, while random tests — backed by a scoreboard, since you can’t hand-predict every random outcome — close out broad functional coverage and surface the bugs a fixed set of directed tests would never have found.